

核心速览

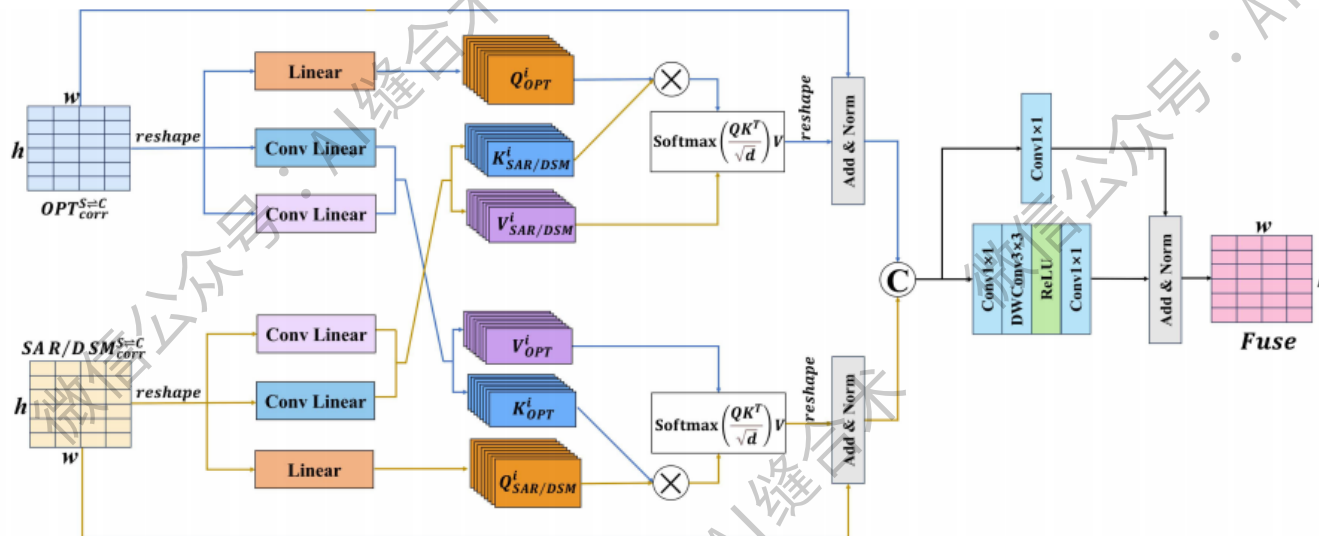


Fig. 4. FFM effectively extracts pertinent information across modalities for global interaction, comprehensively capturing the intricate nonlinear correlations within multimodal data.

论文题目: CFFormer: A Cross-Fusion Transformer Framework for the Semantic Segmentation of Multisource Remote Sensing Images

中文题目: CFFormer: 一种用于多源遥感图像语义分割的交叉融合 Transformer 框架

论文链接: <https://ieeexplore.ieee.org/document/10786275>

github: <https://github.com/masurq/CFFormer>

所属单位: 中国矿业大学（徐州）环境与空间信息学院

核心速览: 本文介绍了一种名为 CFFormer 的跨融合 Transformer 框架，用于多源遥感图像（RSIs）的语义分割。该框架通过并行双流结构和特征校正模块（FCM）来减少不同模态间的差异和噪声，并利用多头交叉注意力机制在特征融合模块

(FFM) 中全局交互和融合不同模态的特征，以充分利用多源 RSIs 的互补信息。

多头交叉注意力特征融合模块 (FFM) : FFM 利用多头交叉注意力机制，在全局范围内交互并融合不同模态的特征，充分利用多源 RSIs 的互补信息。

```
FeatureFusion(  
  (cross): FeatureInteraction(  
    (channel_proj1): Linear(in_features=32, out_features=64, bias=True)  
    (channel_proj2): Linear(in_features=32, out_features=64, bias=True)  
    (act1): ReLU(inplace=True)  
    (act2): ReLU(inplace=True)  
    (cross_attn): CrossAttention(  
      (q1): Linear(in_features=32, out_features=32, bias=False)  
      (kv1): Linear(in_features=32, out_features=64, bias=False)  
      (q2): Linear(in_features=32, out_features=32, bias=False)  
      (kv2): Linear(in_features=32, out_features=64, bias=False)  
      (sr1): Conv2d(32, 32, kernel_size=(5, 5), stride=(4, 4), padding=(2, 2), groups=32)  
      (norm1): LayerNorm((32,), eps=1e-05, elementwise_affine=True)  
      (sr2): Conv2d(32, 32, kernel_size=(5, 5), stride=(4, 4), padding=(2, 2), groups=32)  
      (norm2): LayerNorm((32,), eps=1e-05, elementwise_affine=True)  
    )  
    (end_proj1): Linear(in_features=64, out_features=32, bias=True)  
    (end_proj2): Linear(in_features=64, out_features=32, bias=True)  
    (norm1): LayerNorm((32,), eps=1e-05, elementwise_affine=True)  
    (norm2): LayerNorm((32,), eps=1e-05, elementwise_affine=True)  
  )  
  (channel_emb): ChannelEmbed(  
    (residual): Conv2d(64, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)  
    (channel_embed): Sequential(  
      (0): Conv2d(64, 32, kernel_size=(1, 1), stride=(1, 1))  
      (1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32)  
      (2): ReLU(inplace=True)  
      (3): Conv2d(32, 32, kernel_size=(1, 1), stride=(1, 1))  
      (4): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    )  
    (norm): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  )  
)
```

微信公众号: AI缝合术!

```
Input shape (x1): torch.Size([1, 32, 128, 128])  
Input shape (x2): torch.Size([1, 32, 128, 128])  
Output shape: torch.Size([1, 32, 128, 128])
```